

```

sigma = 0.2
mu    = 0.01
S0, E0, I0, R0 = 990, 9, 1, 0

beta_vals = np.linspace(0.1e-3, 0.5e-3, 9)
gamma_vals = np.linspace(0.05, 0.2, 9)

peak_I = np.zeros((len(gamma_vals), len(beta_vals)))
total_inf = np.zeros_like(peak_I)

for i, gamma in enumerate(gamma_vals):
    for j, beta in enumerate(beta_vals):
        t, S, E, I, R, _ = simulate_2(beta, gamma, sigma, mu, S0, E0, I0,
R0, t_span=(0, 365))
        peak_I[i, j] = np.max(I)
        incidence = beta * S * I
        total_inf[i, j] = np.trapz(incidence, t)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.8),
constrained_layout=True)

B_plot = beta_vals * 1e3
G_plot = gamma_vals

B_mesh, G_mesh = np.meshgrid(B_plot, G_plot)

im0 = axes[0].pcolormesh(B_mesh, G_mesh, peak_I,
shading="auto")
axes[0].set_xlabel(r"$\beta$ ( $\times 10^{-3}$ )")
axes[0].set_ylabel(r"$\gamma$")
axes[0].set_title("Peak infections $\max I(t)$ over 365 days")
cb0 = fig.colorbar(im0, ax=axes[0])
cb0.set_label("Individuals")

im1 = axes[1].pcolormesh(B_mesh, G_mesh, total_inf,
shading="auto")
axes[1].set_xlabel(r"$\beta$ ( $\times 10^{-3}$ )")
axes[1].set_ylabel(r"$\gamma$")
axes[1].set_title("Total infections")
cb1 = fig.colorbar(im1, ax=axes[1])
cb1.set_label("Individuals over 365 days")

plt.show() my code looks redundant, can u optimize it? ignore the
curretn function code they are correct

```

Thought for 9s >

You're right that the structure is a bit repetitive, but the main cost is still `simulate_2`, so the best "optimization" is mostly cleaning things up and avoiding duplicated logic, not heavy vectorization (which is tricky with ODE solves).

Here's a cleaned-up, slightly more efficient version with less redundancy and clearer structure:

python

Copy code

```

import numpy as np
import matplotlib.pyplot as plt

```

```

# Parameters
sigma = 0.2
mu     = 0.01
S0, E0, I0, R0 = 990, 9, 1, 0
init_state = (S0, E0, I0, R0)

beta_vals = np.linspace(0.1e-3, 0.5e-3, 9)
gamma_vals = np.linspace(0.05, 0.2, 9)

# Create grids for plotting
B_plot = beta_vals * 1e3
G_plot = gamma_vals
B_mesh, G_mesh = np.meshgrid(B_plot, G_plot)

# Preallocate result arrays with same shape as mesh
peak_I = np.empty_like(B_mesh)
total_inf = np.empty_like(B_mesh)

# Parameter sweep
for i, gamma in enumerate(gamma_vals):
    for j, beta in enumerate(beta_vals):
        t, S, E, I, R, _ = simulate_2(
            beta, gamma, sigma, mu, *init_state, t_span=(0, 365)
        )

        incidence = beta * S * I
        peak_I[i, j] = I.max()
        total_inf[i, j] = np.trapz(incidence, t)

# Plotting helper
fig, axes = plt.subplots(1, 2, figsize=(12, 4.8), constrained_layout=True)

plots = [
    (peak_I,
     r"Peak infections $\max I(t)$ over 365 days",
     "Individuals"),
    (total_inf,
     "Total infections",
     "Individuals over 365 days"),
]

for ax, Z, title, cbar_label in zip(axes, *zip(*plots)):
    im = ax.pcolormesh(B_mesh, G_mesh, Z, shading="auto")
    ax.set_xlabel(r"$\beta$ (x$10^{-3}$)")
    ax.set_ylabel(r"$\gamma$")
    ax.set_title(title)

    cbar = fig.colorbar(im, ax=ax)
    cbar.set_label(cbar_label)

plt.show()

```

What changed / why it's a bit nicer:

- **Avoid repeated constants:** grouped  $(S_0, E_0, I_0, R_0)$  into `init_state`.
- **Aligned array shapes:** `peak_I` and `total_inf` directly match `B_mesh` / `G_mesh`.
- **Loop logic is minimal:** just the simulation + metrics inside, nothing duplicated.
- **Plotting is DRY:** one loop configures both subplots instead of copy-pasting labels and colorbar logic.

If you'd like, we can also wrap the sweep in a function like `run_sweep(beta_vals, gamma_vals, ...)` so your main script is just "define params → call sweep → plot".



Is this conversation helpful so far?   

